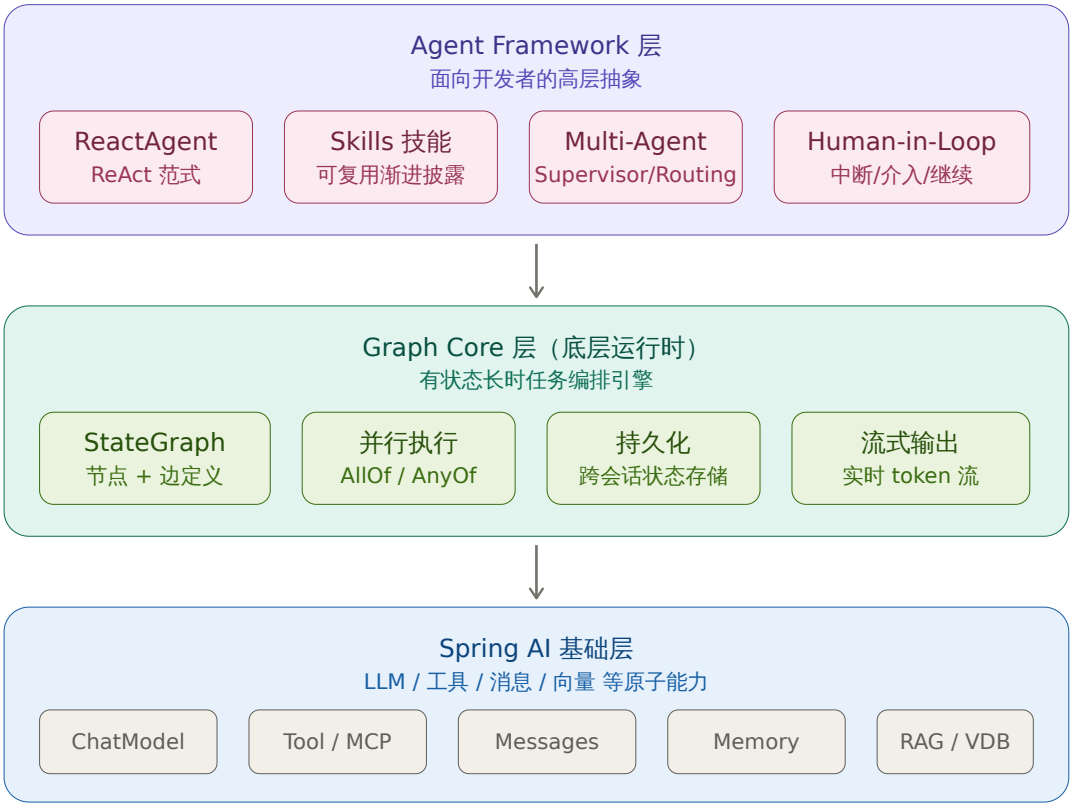
Agent 基础功能搭建

背景需求

随着大模型能力的不断成熟，单轮问答已难以满足复杂业务场景的需求，具备自主推理与工具调用能力的 Agent 成为新的核心诉求。为此，项目在 Spring AI Alibaba 框架的基础上搭建 Agent 基础功能，以 ReactAgent 为核心，整合多模型提供商支持、SSE 流式输出、上下文工程管理与并发控制等能力，为上层业务提供稳定、可扩展的 Agent 执行底座。该模块需支持多轮对话记忆、动态模型配置热更新及工具插拔，兼顾开发效率与生产可靠性，是整个智能对话体系从"模型调用"迈向"自主任务执行"的关键基础设施。

Spring AI Alibaba 里的 Agent

如图所示，Spring AI Alibaba 的 Agent 功能从架构上包含三层：Agent Framework（以 ReactAgent 为核心的 Agent 开发框架）、Graph Core（底层图运行时）和 Spring AI 基础集成层。



ReactAgent

ReAct（Reasoning + Acting）是一种将推理和行动相结合的 Agent 范式。Agent 会循环进行四个步骤：**思考**（分析当前情况，决定下一步）、**行动**（执行工具调用或生成最终答案）、**观察**（接收工具执行结果）、**迭代**（基于观察结果继续思考和行动，直到完成任务）。

Spring AI Alibaba 中的 ReactAgent 基于 **Graph 运行时**构建，Graph 由节点（steps）和边（connections）组成，定义了 Agent 如何处理信息。Agent 在 Graph 中移动，执行三类节点：Model Node（调用 LLM 进行推理决策）、Tool Node（执行工具调用）、Hook Nodes（在关键位置插入自定义逻辑）。

上下文工程（Context Engineering）

上下文工程是以正确的格式提供正确的信息和工具，使 LLM 能够完成任务。缺乏"正确"的上下文是导致 Agent 不可靠的头号原因，Spring AI Alibaba 的 Agent 抽象专门设计用于优化上下文工程。

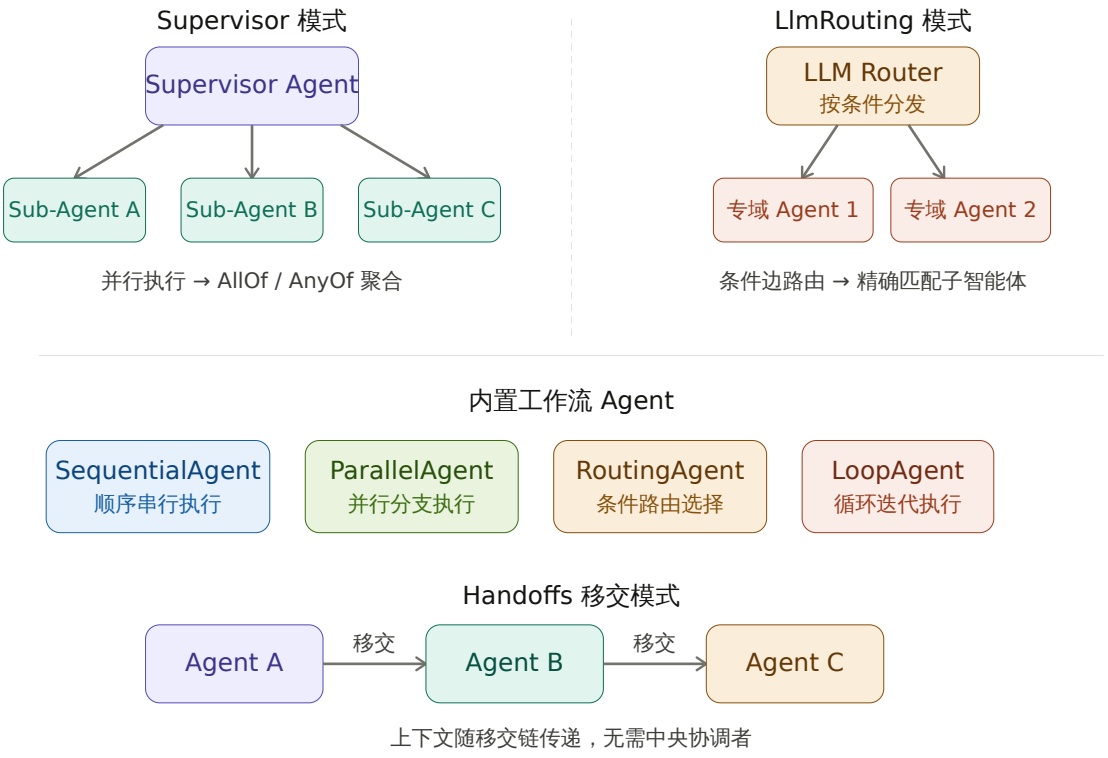
框架将上下文分为三种类型：**模型上下文**（控制每次模型调用包含什么，是瞬态的）、**工具上下文**（工具可以访问和产生什么，持久化到状态）、**生命周期上下文**（模型和工具调用之间发生什么，如摘要、防护栏、日志，也是持久的）。

数据源上分为三种：**运行时上下文**（用户ID、API密钥等静态配置，会话范围）、**State 短期记忆**（当前消息、工具结果，会话范围）、**Store 长期记忆**（用户偏好、历史数据，跨会话）。

实现机制上，`Hook` 和 `Interceptor` 是实现上下文工程的核心手段，允许在 Agent 生命周期的任何步骤中插入逻辑，既可以更新上下文，也可以跳转到 Agent 生命周期的不同步骤。

多智能体模式（Multi-Agent）

在 Spring AI Alibaba 中，Supervisor 与 LlmRouting 已支持「一次选择多个子智能体并并行执行」，配合 Graph 的并行条件边与聚合策略（AllOf / AnyOf），可以更自然地实现多路并行与结果汇总。



Graph Core

Spring AI Alibaba Graph 借鉴了 LangGraph 的设计理念，可以理解为 Java 版本的 LangGraph。社区在此基础上增加了大量预配置节点，并简化了 State 定义流程。

Graph Core 的核心能力包括：ReAct Agent 与 Supervisor 等内置 Agent 模型、并行条件边与分支聚合策略（AllOf / AnyOf）、批量 addEdge、interruptAfter Hook（Human-in-Loop）、AgentToolNode 异步工具执行、流式节点完整输出。

Graph 的关键概念：

- **StateGraph**：定义 Agent 的状态机结构，包含节点与转移边
- **State（短期记忆）**：当前会话范围内所有节点共享的状态对象，是 Agent 循环中信息传递的载体
- **Checkpoint / 持久化**：Graph 运行时支持将 State 持久化，实现 Human-in-Loop 后的无缝续跑，以及跨会话记忆
- **并行边**：1.1.2.0 新特性，一个节点可以同时触发多个下游分支并行执行，再通过 AllOf（全部完成）或 AnyOf（任一完成）聚合策略汇总结果。

Human-in-the-Loop

Human-in-the-Loop 通过三类节点实现：扩展节点（AI 模型流式输出）、人工节点（根据用户反馈决定终止或继续）、后续处理节点（基于人工决策继续执行剩余流程）。

这依赖 Graph 的 `interruptAfter` Hook 在指定节点后挂起整个 Graph，等待用户输入，再从断点继续——State 持久化确保中间状态不丢失。

架构设计

功能模块结构

Agent 模块横跨 `fairy-common`、`fairy-integration`、`fairy-service`、`fairy-web` 四个子模块，其功能结构如下：

```
fairy-common
└── data/llm/
    ├── AgentChatDTO          # 请求入参载体
    ├── AgentEventDTO         # SSE 事件数据载体
    └── ModelConfig           # 模型配置（含思考模式参数扩展）

fairy-integration
└── agent/
    ├── AgentProperties       # 全局配置属性（@ConfigurationP
roperties）
    ├── AgentSseEventType     # SSE 事件类型常量定义
    ├── AgentClientBuilder    # ReactAgent 构建入口（整合模型
+ 工具）
    │
    ├── factory/              # 模型工厂层（按提供商创建 ReactA
gent）
    │   ├── BaseAgentModelFactory       # 工厂抽象基类
    │   ├── DashScopeAgentModelFactory  # DashScope 实现（Op
enAI 兼容模式）
    │   ├── DeepSeekAgentModelFactory   # DeepSeek 实现
    │   └── ZhipuAiAgentModelFactory    # 智谱 AI 实现
    │
    ├── manager/              # 模型管理层（Nacos 配置监听与路
由）
    │   ├── AgentModelFactoryManager    # 工厂路由（按 provid
```

```
er 分发创建请求）
    │   └── AgentModelManager          # Nacos 配置监听与模型
缓存
    │
    └── handler/              # 执行层（推理驱动与 SSE 事件推
送）
        ├── AgentHandler                # Agent 推理驱动核心
        ├── AgentConcurrencyLimiter     # 并发控制（Semaphor
e）
        └── AgentConcurrencyException   # 并发超限异常

fairy-service
└── agent/
    └── AgentService          # 业务编排层（构建流程统一入口）

fairy-web
└── controller/
    └── AgentController       # HTTP 接入层，POST /agent/cha
t
```

整体数据流向

```
客户端
  │  POST /agent/chat
  │  RequestBody: { userId, message, agentSessionId?, model
Name?, maxIterations? }
  ▼
AgentController
  │  委托给 AgentService.chat()
  ▼
AgentService
  ├─① resolveMaxIterations()    取请求值与全局上限的较小值
  ├─② AgentClientBuilder.build()
  │     ├── AgentModelManager.createAgent()
  │     │     └── AgentModelFactoryManager.createAgent()
  │     │           └── XxxAgentModelFactory.createAgent()
按 provider 路由
```

```
  │     │                  └── ReactAgent.builder()
  │     │                        .model(chatModel)
构建各提供商 ChatModel
  │     │                        .tools(toolCallbacks)
注入工具集
  │     │                        .build()
  │     └── 返回 ReactAgent 实例
  ├─③ new SseEmitter(0L)        创建 SSE 连接并注册生命周期回调
  └─④ AgentHandler.run()        异步启动推理
        │
        ├── sendStart()          SSE: agent_start
        │
        ├── reactAgent.stream(message, RunnableConfig{threa
dId=agentSessionId})
        │     │  订阅 Reactor 流，逐个处理 NodeOutput
        │     │
        │     ├── AGENT_MODEL_STREAMING
        │     │     ├── reasoningContent 非空 → SSE: agent_
thinking（受 streamThinking 开关控制）
        │     │     └── text 非空          → SSE: agent_ans
wer（分块流式下发）
        │     │
        │     ├── AGENT_MODEL_FINISHED
        │     │     └── hasToolCalls()     → SSE: agent_too
l_call
        │     │
        │     └── AGENT_TOOL_FINISHED
        │           └── toolResponse       → SSE: agent_too
l_result
        │
        └── sendEnd()            SSE: agent_end + [DONE] →
sseEmitter.complete()


客户端接收 SSE 事件序列（示例）:
  agent_start
    → agent_thinking（若开启且模型支持，可出现多次）
    → agent_tool_call（若 Agent 决策调用工具，可出现多次）
```

```
    → agent_tool_result（工具执行完毕，与 agent_tool_call 成对出现）
    → agent_answer（最终答案，分块流式，可出现多次）
    → agent_end
    → [DONE]
```

根据上述数据流向示意，其执行过程如下：

1. 客户端向 `POST /agent/chat` 发送包含 `userId`、`message`、可选的 `agentSessionId` 和 `modelName` 等字段的 JSON 请求，由 `AgentController` 接收后直接委托给 `AgentService.chat()`。
2. 然后 `AgentService` 调用 `AgentClientBuilder.build()` 来构建 `ReactAgent`，其内部依次经由 `AgentModelManager` 从 N缓存按模型名取出 `ModelConfig`、`AgentModelFactoryManager`，按 `provider` 字段路由到对应工厂、具体工厂完成 ChatModel 初始化并通过 `ReactAgent.builder().model().tools().saver().build()` 组装出实例。
3. 随后 `AgentService` 创建 `SseEmitter` 并注册生命周期回调，实例化 `AgentHandler` 后调用 `run()` 方法，立即返回 `SseEmitter` 给 HTTP 层。
4. 模型推理通过 `run()` 方法中的 `CompletableFuture.runAsync` 实现异步运行：线程发送 `agent_start` 事件，以 `agentSessionId` 作为 `threadId` 构造 `RunnableConfig`，调用 `reactAgent.stream(message, config)` 订阅 Reactor 流并以 `blockLast()` 阻塞；
5. `handleNodeOutput()` 在流中持续处理每个 `NodeOutput` —— `AGENT_MODEL_STREAMING` 时根据是否携带 `reasoningContent` 分别推送 `agent_thinking` 或 `agent_answer` 事件，`AGENT_MODEL_FINISHED` 时若包含工具调用则推送 `agent_tool_call`，`AGENT_TOOL_FINISHED` 时推送 `agent_tool_result`；
6. 流正常结束后 `sendEnd()` 依次推送 `agent_end` 与 `[DONE]` 并调用 `sseEmitter.complete()` 关闭连接。

功能实现

数据层

AgentChatDTO

```
@Data
public class AgentChatDTO {
```

```
    private String  userId;          // 用户 ID
    private String  message;         // 用户本轮输入的问题
    private String  agentSessionId;  // Agent 会话 ID（多轮对话的上下文标识）
    private String  modelName;       // 指定本次使用的模型名称
    private Integer maxIterations;   // 本次最大 ReAct 循环次数

    public String getAgentSessionId(); // 惰性生成：首次调用时自动生成，格式 agent_{userId前6位}_{UUID前8位}
}
```

`AgentChatDTO` 是 Agent 对话接口的请求入参。其中 `agentSessionId` 是多轮对话的核心——客户端首次请求时不传该字段，由 `getAgentSessionId()` 自动生成并通过 SSE 事件中的 `agentSessionId` 字段返回给客户端。`modelName` 为空时由 `AgentProperties.defaultModel` 兜底，`maxIterations` 为空时由全局配置兜底，使客户端可以只传必填字段 `message` 即可发起一次 Agent 对话。

AgentEventDTO

```
@Data
@Builder
public class AgentEventDTO {

    private String  eventType;       // 事件类型，对应 AgentSseEventType 中的常量
    private String  content;         // 主要文字内容（推理文本、答案分块、错误信息等）
    private String  toolName;        // 工具名称（仅工具相关事件有值）
    private String  toolInput;       // 工具入参 JSON 字符串（仅 agent_tool_call 有值）
    private String  toolResult;      // 工具执行结果（仅 agent_tool_result 有值）
    private String  agentSessionId;  // 会话 ID，所有事件均携带，便于前端按会话聚合展示
    private Integer iterationIndex;  // 当前 ReAct 迭代轮次，从 1 开始
```

```
    private Long    timestamp;       // 事件产生时间戳（毫秒）

    public static AgentEventDTO ofContent(String eventType,
String content, String agentSessionId);
    public static AgentEventDTO ofToolCall(String toolName,
String toolInput, String agentSessionId, int iterationInde
x);
    public static AgentEventDTO ofToolResult(String toolNam
e, String toolResult, String agentSessionId, int iterationI
ndex);
}
```

`AgentEventDTO` 是所有 SSE 事件的统一数据载体，序列化为 JSON 后作为 SSE `data` 字段的值传输。相比普通对话流式接口，Agent 事件额外携带了工具调用信息和推理过程，使前端能够以可视化方式展示 Agent 的完整执行链路。不同类型的事件仅填充与自身相关的字段（例如 `toolName` / `toolInput` 仅在工具调用事件中有值），其余字段为 null。三个静态工厂方法分别针对纯内容事件、工具调用事件、工具结果事件提供快捷构建方式，避免每次手动填充不相关字段。

AgentSseEventType

```
public final class AgentSseEventType {

    private AgentSseEventType() {} // 工具类，禁止实例化

    public static final String AGENT_START       = "agent_s
tart";       // 任务启动
    public static final String AGENT_THINKING    = "agent_t
hinking";    // LLM 推理过程（Thought）
    public static final String AGENT_TOOL_CALL   = "agent_t
ool_call";   // 工具调用决策
    public static final String AGENT_TOOL_RESULT = "agent_t
ool_result"; // 工具执行结果
    public static final String AGENT_ANSWER      = "agent_a
nswer";      // 最终答案（流式分块）
    public static final String AGENT_END         = "agent_e
nd";         // 任务完成
    public static final String AGENT_ERROR       = "agent_e
rror";       // 执行异常
```

```
    public static final String DONE              = "[DON
E]";            // 流结束标记
}
```

`AgentSseEventType` 定义了 Agent 模块专属的 SSE 事件类型常量集合，通过 SSE 协议的 `event` 字段加以区分。 `AGENT_THINKING` 是 Agent 与普通对话最重要的视觉区分点，它将 LLM 的思维链（Chain-of-Thought）实时暴露给前端，仅在配置项 `streamThinking = true` 且所用模型支持思考模式（如 qwen3.5-plus）时才会产生；`AGENT_TOOL_CALL` 和 `AGENT_TOOL_RESULT` 则让用户能观察到 Agent 调用了哪些工具及工具的返回结果，体现出 Agent 能力相比普通问答的核心差异。

配置层

AgentProperties

```
@Data
@Component
@ConfigurationProperties(prefix = "fairy.agent")
public class AgentProperties {

    private String  defaultModel   = "qwen3.5-plus"; // 未
指定模型时的全局默认值
    private int     maxIterations  = 10;             // ReA
ct 最大循环次数全局上限
    private boolean streamThinking = true;           // 是
否向客户端推送推理过程（Thought）
    private int     maxHistorySize = 10;             // Mem
orySaver 保留的最大历史轮次

    private Concurrency concurrency = new Concurrency(); //
并发控制配置块

    @Data
    public static class Concurrency {
        private int  maxConcurrent    = 20;   // Semaphore
最大许可数（即最大并发请求数）
        private long acquireTimeoutMs = 5000; // 等待获取许可
的超时时间（毫秒）
```

```
    }
}
```

`AgentProperties` 通过 `@ConfigurationProperties` 将 `application.yml` 中 `fairy.agent.*` 的所有配置项绑定为强类型对象。`defaultModel` 和 `maxIterations` 控制推理行为的默认值；`streamThinking` 控制是否将 LLM 的思考过程实时推送给客户端，可按需关闭以降低 SSE 传输量，支持在 Nacos 动态配置变更时无需重启服务即可调整并发上限。

工厂层

BaseAgentModelFactory

```
public abstract class BaseAgentModelFactory {

    public abstract boolean supports(String provider);  //
判断是否支持指定的提供商标识
    public abstract String getProviderName();            //
返回本工厂负责的提供商名称
    public abstract ReactAgent createAgent(ModelConfig mode
lConfig,
                                           ToolCallback[] t
ools,
                                           BaseCheckpointSa
ver saver); // 创建 ReactAgent
}
```

`BaseAgentModelFactory` 是所有模型工厂的抽象基类，定义了工厂体系的统一契约。每个具体子类负责一个 AI 提供商的 SDK 初始化细节（API Key 读取、BaseURL 配置、请求参数映射、思考模式扩展等），并最终通过 `ReactAgent.builder()` 组装出可执行的 Agent 实例。`createAgent()` 接收 `BaseCheckpointSaver` 参数（即 MemorySaver 实例），要求所有工厂创建的 Agent 都必须挂载全局共享的 Checkpoint 存储，确保相同 `threadId` 下的多次请求能够共享会话历史。Agent 工厂体系与现有 Chat 模块的工厂体系完全独立，不共享代码，互不干扰。

DashScopeAgentModelFactory

```
@Slf4j
@Component
```

```
public class DashScopeAgentModelFactory extends BaseAgentMo
delFactory {

    private static final String COMPATIBLE_BASE_URL = "http
s://dashscope.aliyuncs.com/compatible-mode/v1";

    @Override public boolean supports(String provider);
    @Override public String getProviderName();           //
返回 "dashscope"
    @Override public ReactAgent createAgent(ModelConfig mod
elConfig, ToolCallback[] tools, BaseCheckpointSaver saver);
}
```

DashScope 工厂使用 `OpenAiApi + OpenAiChatModel`，而非 Spring AI Alibaba 提供的 `DashScopeChatModel`。原因在于 `DashScopeChatModel` 内部固定调用 DashScope 的 `text-generation` 接口，而部分模型（如 qwen3.5-plus）属于多模态模型，需要走 `multimodal-generation` 接口，两者 URL 不兼容，直接使用 `DashScopeChatModel` 会导致请求报错。通过切换为 DashScope 的 OpenAI 兼容端点（`/compatible-mode/v1`），可以用标准 OpenAI 客户端访问 DashScope 全系列模型。思考模式参数（`enable_thinking`、`thinking_budget`）通过 `OpenAiChatOptions.builder().extraBody(map)` 方式注入请求体，这是 Spring AI 1.1.2 中替代已移除的 `additionalModelRequestFields()` 的正确方式。

DeepSeekAgentModelFactory

```
@Slf4j
@Component
public class DeepSeekAgentModelFactory extends BaseAgentMod
elFactory {

    @Override public boolean supports(String provider);
    @Override public String getProviderName();           //
返回 "deepseek"
    @Override public ReactAgent createAgent(ModelConfig mod
elConfig, ToolCallback[] tools, BaseCheckpointSaver saver);
}
```

DeepSeek 工厂使用 `DeepSeekApi + DeepSeekChatModel` 构建 ChatModel，再通过 `ReactAgent.builder()` 组装为 Agent 实例。`ModelConfig` 中的 `apiKey`、`baseUrl`、

`parameters`（temperature、maxTokens 等）由 Nacos 动态下发，工厂负责将这些配置映射到 DeepSeek SDK 的对应初始化参数。

ZhipuAiAgentModelFactory

```
@Slf4j
@Component
public class ZhipuAiAgentModelFactory extends BaseAgentMode
lFactory {

    @Override public boolean supports(String provider);
    @Override public String getProviderName();           //
返回 "zhipuai"
    @Override public ReactAgent createAgent(ModelConfig mod
elConfig, ToolCallback[] tools, BaseCheckpointSaver saver);
}
```

智谱 AI 工厂使用 `ZhiPuAiApi + ZhiPuAiChatModel` 构建 ChatModel，逻辑结构与 DeepSeek 工厂一致。

上述三个具体工厂（DashScope / DeepSeek / ZhipuAi）共同构成当前支持的提供商集合，新增提供商时只需新增一个 `@Component` 子类，无需修改任何已有代码，符合开闭原则。

模型管理层

AgentModelFactoryManager

```
@Slf4j
@Component
public class AgentModelFactoryManager {

    private final Map<String, BaseAgentModelFactory> factor
yMap; // 以 provider 名称为 key 的工厂注册表

    @Autowired
    public AgentModelFactoryManager(List<BaseAgentModelFact
ory> factories); // 启动时收集所有工厂 Bean
```

```
    public ReactAgent createAgent(ModelConfig modelConfig,
ToolCallback[] toolCallbacks, BaseCheckpointSaver saver);
    public boolean isProviderSupported(String provider);
    public List<String> getSupportedProviders();
}
```

`AgentModelFactoryManager` 是工厂路由层，职责是根据 `ModelConfig.provider` 字段将 Agent 创建请求分发给正确的工厂实现。它在构造时通过 Spring 的列表注入机制自动收集所有 `BaseAgentModelFactory` 实现的 Bean，以 `provider` 名称为 key 存入 `ConcurrentHashMap`。`createAgent()` 执行时先校验 provider 合法性，再委托对应工厂完成实际创建；若 provider 不在注册表中，抛出明确的异常并列出当前所有可用提供商，便于排查配置错误。

AgentModelManager

```
@Slf4j
@Component
@RequiredArgsConstructor
public class AgentModelManager {

    private final NacosConfigManager      nacosConfigManage
r;
    private final AgentModelFactoryManager factoryManager;

    private static final String AGENT_DATA_ID = "fairy_agen
t_config"; // Nacos DataId
    private static final String GROUP_ID      = "FAIRY_LLM_
GROUP";    // Nacos Group

    private final Map<String, ModelConfig> agentModels = ne
w ConcurrentHashMap<>(); // 模型配置缓存

    @PostConstruct
    public void init();
// 初始化：拉取配置 + 注册 Nacos 监听器

    private Set<String> loadModels(String configJson);
// 解析 JSON 写入缓存，返回本次 key 集合
```

```
    public ReactAgent createAgent(String modelName, ToolCal
lback[] toolCallbacks, BaseCheckpointSaver saver);
}
```

`AgentModelManager` 是 Agent 模块的模型配置管理中枢，监听Nacos 的 DataId。`init()` 在 Spring 容器启动完成后执行：首先主动拉取一次 Nacos 上的模型配置 JSON，解析后填入 `agentModels` 缓存；随后注册 Nacos 配置变更监听器，配置更新时自动重新解析并刷新缓存（同时清除已下线的模型），实现模型配置的动态热更新，无需重启服务。`createAgent()` 按模型名称从缓存中取出 `ModelConfig`，委托给 `AgentModelFactoryManager` 完成 Agent 实例创建。

构建层

AgentClientBuilder

```
@Slf4j
@Component
@RequiredArgsConstructor
public class AgentClientBuilder {

    private final AgentModelManager      agentModelManager;
    private final ToolCallbackProvider   toolCallbackProvid
er;
    private final AgentProperties        agentProperties;
    private final MemorySaver            memorySaver;
// 全局共享，按 threadId 隔离会话

    public ReactAgent build(String modelName, String sessio
nId); // 构建并返回 ReactAgent 实例
}
```

`AgentClientBuilder` 是 ReactAgent 实例的统一构建入口，负责整合三类资源：**模型**（通过 `AgentModelManager` 按名称创建）、**工具**（通过 `ToolCallbackProvider` 获取已注册的全量工具回调）。`build()` 方法对调用方屏蔽了模型解析、工厂路由、工具注入的内部细节，调用方只需提供 `modelName` 和 `sessionId`，即可拿到一个开箱即用的 `ReactAgent` 实例。

实现层

AgentHandler

```
@Slf4j
@RequiredArgsConstructor
public class AgentHandler {

    private final ReactAgent              reactAgent;
// 已配置好模型、工具、MemorySaver 的 Agent 实例
    private final SseEmitter              sseEmitter;
// 当前请求的 SSE 连接
    private final AgentChatDTO            agentChatDTO;
// 请求参数（含 sessionId、message 等）
    private final AgentProperties         agentProperties;
// 全局配置（streamThinking 等开关）
    private final AgentConcurrencyLimiter concurrencyLimite
r;  // 推理结束后释放许可使用

    private Integer       chunkId             = 1;
// SSE 事件序号，单调递增
    private StringBuilder assistantTextBuffer = new StringB
uilder(); // 累积本轮 Agent 回复文本

    public void run(); // 执行入口

    private void executeAgentStream();
// 订阅 ReactAgent 流，阻塞至结束
    private void handleNodeOutput(NodeOutput output) throws
Exception; // 按 OutputType 映射 SSE 事件
    private void sendStart() throws Exception;
    private void sendEnd();
    private void handleConcurrencyRejected(AgentConcurrency
Exception e); // 推送 429 事件并关闭连接
    private void handleError(Throwable e);
    private void sendSseEvent(String event, String data) th
rows Exception;
}
```

`AgentHandler` 是 Agent 执行链路的核心驱动器，每次请求由 `AgentService` 手动 `new` 出来，持有本次请求的 ReactAgent 实例和 SseEmitter。`run()` 方法是模型调用的执行入口。

`assistantTextBuffer` 用于存储本轮对话的 Agent 完整回答，`handleNodeOutput()` 将每个 `AGENT_MODEL_STREAMING` 事件的文本片段实时推送给客户端的同时追加到 buffer。

服务层与 Web 层

AgentService

```
@Slf4j
@Service
@RequiredArgsConstructor
public class AgentService {

    private final AgentClientBuilder      agentClientBuilde
r;
    private final AgentProperties         agentProperties;
    private final AgentConcurrencyLimiter concurrencyLimite
r;

    public SseEmitter chat(AgentChatDTO agentChatDTO); //
对外唯一入口，返回 SseEmitter

    private int  resolveMaxIterations(Integer requested);
// 取请求值与全局上限的最小值
    private void setSseCallbacks(SseEmitter emitter, String
agentSessionId); // 注册生命周期回调
}
```

`AgentService` 负责将 Agent 对话启动流程的各环节串联起来：① 解析最终迭代次数上限，② 构建 ReactAgent 实例，③ 创建 SseEmitter 并注册 completion / timeout / error 三类生命周期回调，④ 实例化 `AgentHandler` 并调用 `run()` 方法启动异步推理。

`resolveMaxIterations()` 对客户端传入的迭代次数进行上限裁剪，防止恶意或误操作的超大值导致 LLM 反复调用，造成不必要的 token 消耗。

AgentController

```
@Slf4j
@RestController
@RequiredArgsConstructor
@RequestMapping("/agent")
public class AgentController {

    private final AgentService agentService;

    @PostMapping(path = "/chat", produces = MediaType.TEXT_
EVENT_STREAM_VALUE)
    public SseEmitter agentChat(@RequestBody AgentChatDTO a
gentChatDTO); // POST /agent/chat
}
```

`AgentController` 是 Agent 模块对外暴露的 HTTP 入口。`produces = TEXT_EVENT_STREAM_VALUE` 声明响应类型为 SSE 流，Spring MVC 会自动设置 `Content-Type: text/event-stream` 等 HTTP 响应头。